Cliente	Bidaia Go!
Proyecto	Proyecto React BootCamp 2026

Grupo	4
Fecha	11.05.2026
Version	1.0

Memoria Técnica del Proyecto React

BidaiaGo

Integrantes del grupo

- Jefferson Aguilar
- Asier González
- David López
- Montse García

INDICE

1. Introducción
2. Objetivos del Proyecto
3. Tecnologías Utilizadas
4. Metodología de Trabajo
5. Estructura del Proyecto
6. Funcionalidades Implementadas
7. Uso de Hooks
8. Uso de localStorage
9. Componentes Implementados
10. Organización del Desarrollo y Trabajo por Features
11. Diseño y Estilos
12. Problemas Encontrados
13. Mejoras Futuras
14. Conclusión

1. Introducción

BidaiaGo es una aplicación web desarrollada con React, TypeScript y Vite orientada a la consulta de información útil de paises para viajeros.

El objetivo principal del proyecto ha sido crear una plataforma dinámica capaz de consumir APIs externas para mostrar información actualizada sobre países, clima y recomendaciones de viaje, aplicando los conocimientos adquiridos durante el Bootcamp FullStack.

El desarrollo del proyecto se realizó en un periodo de **7 días**, trabajando mediante ramas en GitHub para facilitar el desarrollo colaborativo y la integración de funcionalidades.

2. Objetivos del Proyecto

Los objetivos principales del proyecto fueron:

- Consumir información desde APIs externas.
- Aplicar React mediante componentes funcionales.
- Utilizar hooks para gestión de estado y efectos.
- Implementar almacenamiento local mediante localStorage.
- Organizar una arquitectura escalable de componentes y servicios.
- Aplicar navegación mediante React Router DOM.
- Crear una interfaz responsive y reutilizable.

3. Tecnologías Utilizadas

Tecnología	Versión	Uso
React	19.2.5	Librería principal
TypeScript	6.0.3	Tipado estático
Vite	8.0.10	Entorno de desarrollo
React Router DOM	7.15.0	Navegación entre páginas
Axios	1.16.0	Peticiones HTTP
Framer Motion	12.38.0	Animaciones
CSS	—	Estilos y diseño responsive
GitHub	—	Control de versiones

4. Metodología de Trabajo

El proyecto se organizó mediante ramas de GitHub para trabajar distintas funcionalidades de forma separada y facilitar posteriormente la integración en una rama común.

También se utilizó ChatGPT como apoyo técnico durante el desarrollo, especialmente para:

- resolución de errores,
- consulta de documentación,
- optimización de lógica,
- apoyo en estructura de componentes y consumo de APIs.

La comunicación y coordinación del equipo se realizó de forma colaborativa durante todo el desarrollo.

Repositorio del proyecto:

[Repositorio BidaiaGo en GitHub]

5. Estructura del Proyecto

El proyecto se organizó siguiendo una estructura modular para facilitar el mantenimiento y la escalabilidad.

```
src/
├── components/
├── pages/
├── services/
├── styles/
├── hooks/
├── models/
├── App.tsx
└── main.tsx
```

Organización principal

Components

Contiene componentes reutilizables como:

- Header
- Footer
- Formularios
- Elementos de navegación

Pages

Incluye las páginas principales:

- HomePage
- AboutPage
- TripListPage
- DestinationDetailPage
- TipsViajesPage

Services

Gestiona el consumo de APIs externas

Hooks

Incluye hooks personalizados como:

- useLocalStorage

Styles

Separación de estilos globales, de componentes y páginas.

6. Funcionalidades Implementadas

6.1 Sistema de Navegación

Se implementó navegación mediante React Router DOM.

Características:

- rutas dinámicas,
- navegación entre páginas,
- integración de enlaces en el Header

6.2 Página “Tips Viajes”

Funcionalidad principal del proyecto.

Permite buscar países y consultar:

- bandera,
- capital,
- población,
- idiomas,
- moneda,
- zona horaria,
- lado de conducción,

- clima actual,
- recomendaciones de seguridad.

6.3 Consumo de APIs

El proyecto utiliza APIs externas para obtener información actualizada.

APIs utilizadas

API	Uso
Rest Countries	Información de países
Open-Meteo	Información meteorológica
Open-Meteo Geocoding	Coordenadas geográficas
Pexels	Imágenes (busqueda según pais)

7. Uso de Hooks

useState

Utilizado para:

- almacenar resultados de búsqueda,
- controlar formularios,
- gestionar estados de carga y errores.

useEffect

Utilizado para:

- llamadas a APIs,
- renderizado dinámico de datos,
- actualización de información.

Hook personalizado: useLocalStorage

Implementado para almacenar información persistente en el navegador.

8. Uso de localStorage

Se utilizó localStorage para mantener datos del usuario entre sesiones y mejorar la experiencia de uso.

Esto permitió:

- persistencia de información,
- mantenimiento de estados,
- reutilización de datos sin necesidad de nuevas consultas.

9. Componentes Implementados

El proyecto supera el requisito mínimo de cinco componentes funcionales.

Entre ellos:

- Header
- Footer
- ContactForm
- TipsViajesPage
- DestinationDetailPage
- TripListPage
- HomePage

Cada componente fue diseñado con una responsabilidad específica y reutilizable.

10. Organización del Desarrollo y Trabajo por Features

El desarrollo del proyecto se organizó mediante una estrategia de ramas orientada a features, permitiendo que cada integrante trabajara funcionalidades independientes sin interferir directamente en el trabajo del resto del equipo.

La rama principal de integración durante el desarrollo fue:

feat/commons

A partir de esta rama se generaron distintas ramas específicas para cada funcionalidad.

10.a Desarrollo de la Feat/dashboard

Una de las principales funcionalidades desarrolladas fue la creación del dashboard principal de exploración de destinos.

El dashboard se diseñó como pantalla principal de uso de la aplicación, permitiendo al usuario:

- explorar destinos,
- buscar países,
- filtrar resultados por región,
- visualizar información resumida de cada destino.

Decisiones Técnicas

Durante el desarrollo se tomaron distintas decisiones para simplificar la integración y mantener la estabilidad del proyecto.

Reutilización de HomePage

Esta decisión evitó:

- renombrados innecesarios,
- conflictos de imports,
- modificaciones adicionales de rutas.

El objetivo fue priorizar un enfoque pragmático orientado al MVP.

La pantalla principal se estructuró mediante:

- buscador de países,
- filtro por región,
- grid responsive de cards,
- estados de carga,
- control de errores,
- estado sin resultados.

Esta estructura permitió:

- reducir conflictos de integración,
- mantener una base común estable,
- desarrollar funcionalidades de forma paralela,
- facilitar revisiones mediante Pull Requests.

10.b Desarrollo de la Feat/auth

La autenticación tuvo como objetivo incorporar un sistema básico de gestión de usuarios dentro de la aplicación BidaiaGo, permitiendo el registro, inicio de sesión.

Esta funcionalidad fue desarrollada siguiendo un enfoque MVP (Minimum Viable Product), priorizando la persistencia local y la integración con la navegación principal de la aplicación.

- implementación de login y register,
- integración de rutas de autenticación en el navbar.

La implementación de autenticación buscaba:

- permitir registro de usuarios,
- gestionar inicio de sesión,
- mantener persistencia de sesión,
- crear una estructura base para futuras funcionalidades de usuario,
- integrar la autenticación dentro de la navegación principal.

Arquitectura Implementada

Debido al alcance actual del proyecto, no se desarrolló backend propio ni

sistema de autenticación externo.

La gestión de usuarios se implementó utilizando:

- React Context API,
- estado global mediante AuthContext,
- persistencia local usando localStorage.

Esto permitió simular un sistema funcional de autenticación manteniendo la arquitectura preparada para futuras integraciones con backend.

Persistencia de Usuarios

Los usuarios registrados se almacenan localmente en el navegador utilizando localStorage.

Cada usuario contiene información básica como:

- id
- name
- email
- password

Uso de AuthContext

Se implementó un contexto global de autenticación mediante AuthContext.

Este contexto centraliza:

- usuario autenticado,
- login,
- registro,
- logout,
- actualización de datos del usuario.

El uso de Context API permitió evitar prop drilling y facilitó compartir el estado autenticado entre distintos componentes de la aplicación.

10.c Desarrollo de la Feat/detail

Otra de las funcionalidades principales desarrolladas en el proyecto fue la

implementación de la vista de detalle de destinos.

Esta feature tuvo como objetivo ampliar la información mostrada al usuario sobre cada país, integrando datos enriquecidos relacionados con clima, seguridad y características generales del destino.

La pantalla de detalle se diseñó para permitir al usuario consultar información ampliada de un país seleccionado desde el dashboard principal.

Entre los datos mostrados se incluyeron:

- nombre común y oficial,
- capital,
- región y subregión,
- población,
- área,
- idiomas,
- monedas,
- banderas,
- zonas horarias,
- información meteorológica,
- recomendaciones de seguridad,
- enlaces a mapas.

Gestión Meteorológica

La aplicación obtiene información meteorológica actual de la capital del país seleccionado.

Proceso implementado:

1. Obtención de coordenadas mediante OpenStreetMap.
2. Consulta meteorológica usando Open-Meteo.
3. Transformación de códigos meteorológicos a descripciones legibles.
4. Asociación de iconos climáticos dinámicos.

Entre los datos mostrados:

- temperatura,
- humedad,
- velocidad del viento,

- estado del clima,
- iconografía meteorológica.

10.d Desarrollo de la Feat/contact

En esta rama se ha desarrollado la página de contacto. Esta página permite a los usuarios enviar consultas, sugerencias o incidencias a través de un formulario, almacenando los mensajes en el localStorage del navegador. También incluye información de contacto de la empresa (email, teléfono, dirección, horario).

Definir las interfaces y tipos que garantizan la integridad de los datos en toda la funcionalidad de contacto.

Contenido:

Tipo	Descripción
ContactFormData	Datos que introduce el usuario en el form
FormErrors	Errores de validación de cada campo
StoredMessage	Mensaje completo guardado en localStorage (hereda de ContactFormData e incluye id y date)

Encapsula toda la lógica del formulario en un **hook** personalizado, separando la lógica de negocio de la capa de presentación.

Hooks de React utilizados:

- useState: para gestionar formData, errors, isSubmitting, success
- useEffect: no ha sido necesario en este caso

Funciones principales:

Función	Descripción
validate	Valida que los campos no estén vacíos y que el email tenga formato correcto
handleChange	Actualiza el campo del formulario y limpia su error específico
resetForm	Limpia todo el formulario
saveToLocalStorage	Guarda el mensaje en localStorage con id único y fecha ISO
handleSubmit	Valida, simula envío asíncrono (setTimeout), guarda y resetea

Uso de localStorage

- Clave utilizada: "bidaiaGoMessages"
- Formato del dato guardado: array de objetos StoredMessage

Simulación de asincronía: Se utiliza setTimeout con 500ms para simular una llamada a un API real.

11. Diseño y Estilos

Se implementó una interfaz responsive basada en:

- diseño mobile-first,
- variables CSS reutilizables,
- paleta visual inspirada en aplicaciones de viaje y naturaleza.

Además, se organizó el CSS en distintos archivos para mejorar la mantenibilidad.

12. Problemas Encontrados

Durante el desarrollo surgieron distintas dificultades técnicas:

- gestión del estado compartido,
- integración de rutas,
- persistencia de datos,
- estructura inicial del proyecto,
- coordinación de ramas durante el trabajo grupal.

Estos problemas fueron solucionados mediante depuración, reorganización del código y trabajo colaborativo.

13. Mejoras Futuras

Las funcionalidades planteadas para futuras versiones incluyen:

- creación de itinerarios personalizados,
- incorporación de imágenes e iconografía meteorológica,
- mejora visual de tarjetas y experiencia de usuario,
- integración de backend propio.

14. Conclusión

El proyecto BidaiaGo permitió aplicar conocimientos fundamentales de React y TypeScript en un entorno colaborativo y orientado a funcionalidades reales.

Durante el desarrollo se trabajó:

- consumo de APIs,
- arquitectura de componentes,
- persistencia de datos,
- organización mediante ramas,
- trabajo en equipo.

Además del aprendizaje técnico, el proyecto ayudó a mejorar la coordinación grupal, la resolución de errores y la planificación de tareas dentro de un entorno de desarrollo moderno.